Brno University of Technology — FIT

Image Filter Application

ZPOe 25/26 — Applied Image Processing in C++/OpenCV

Adar Otieno • Deniz Aksoy • Eyobed Awel Nuri

May 9, 2026

Outline

1. Project Overview
2. Eyobed's Filters
3. Adar's Filters
4. Deniz's Filters
5. Summary

Project Overview

Goal

Design and implement **12 image processing filters** applied to human face images. Filters are built manually using pixel-level C++ code — no high-level OpenCV abstractions.

Tech Stack

- **Language:** C++17
- **Library:** OpenCV (HighGUI, core)
- **Build:** CMake

Contribution Split

- 4 filters per student
- Shared interactive GUI
- Unified `Filter` struct interface

Architecture

Each filter is registered as a function pointer inside a `Filter` struct carrying author, name, description, and the `apply()` callback.

Interactive Application

- Built on **OpenCV HighGUI**
- **Two trackbars:**
  - Image selector (iterates `data/` directory)
  - Filter selector (12 entries)
- Side-by-side 800×600 preview window
- Dynamic metadata panel (filter name, author, description)

Filter Interface

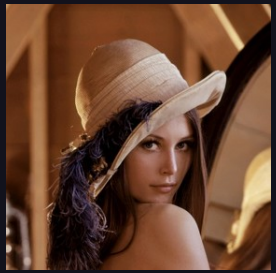
```
struct Filter {
  string author;
  string name;
  string description;
  Mat (*apply)(const Mat&);
};
```

Running

```
mkdir build && cd build
cmake ..  && make
./project
```

Input Images

Four face images used as the common test dataset









Eyobed's Filters — Overview

[Hist Eq] Histogram Equalization

Redistributes luminance values via a CDF remap in YCrCb space. Boosts contrast without color shift.

[Post] Posterize

Quantizes each BGR channel into $L = 8$ levels using a precomputed LUT. $O(W \times H)$.

[Lens] Lens Distortion

Inverse geometric mapping with radial coefficient k. Simulates barrel distortion.

[Vig] Vignette Blur

15×15 box blur blended with original via a radial distance mask $d \in [0, 1]$.

Histogram Equalization

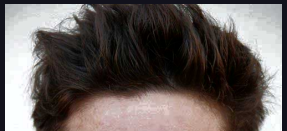
Algorithm:

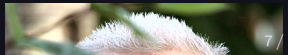
1. Convert BGR $\rightarrow$ YCrCb
2. Compute PDF: $PDF(i) = \dfrac{\text{count}(i)}{W \cdot H}$
3. Compute CDF:
$$CDF(i) = \sum_{j=0}^{i} PDF(j)$$
4. Remap:
$\text{Out}(x, y) = \text{round}(CDF \times 255)$
5. Convert YCrCb $\rightarrow$ BGR

Complexity: $O(W \times H)$









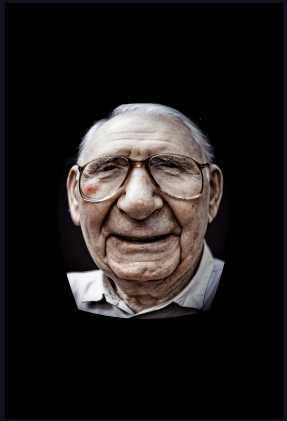
Lens Distortion

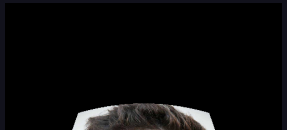
Inverse Geometric Mapping:

1. Normalize: $d_x = \frac{x - c_x}{R}$, $d_y = \frac{y - c_y}{R}$
2. Distort: $\text{dist} = 1 + k(d_x^2 + d_y^2)$
   Positive $k \rightarrow$ barrel distortion
3. Map back: $s_x = d_x \cdot \text{dist} \cdot R + c_x$

Nearest-neighbor interpolation.
$O(W \times H)$.







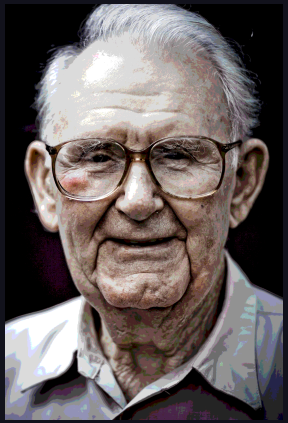
Posterize

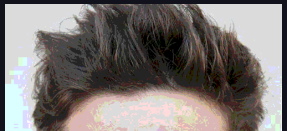
LUT-based Quantization ($L = 8$):

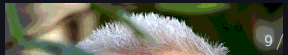
1. Step: $\text{step} = \dfrac{255}{L-1} \approx 36.4$
2. Build LUT: $\text{level}_i = \text{round}(i/\text{step})$
   $\text{LUT}[i] = \text{round}(\text{level}_i \times \text{step})$
3. Apply: $\text{Out}_c(x, y) = \text{LUT}[\text{In}_c(x, y)]$

$O(W \times H) + O(256)$ for LUT.









Vignette Blur

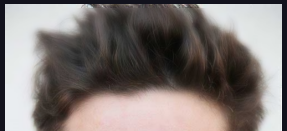
Two-Stage Radial Blend:

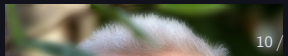
1. Box blur (15×15 kernel):
   $\text{Blur} = \frac{1}{225}\sum_{dy,dx=-7}^{7} \text{In}(x+dx, y+dy)$
2. Radial mask:
   $d = \frac{\sqrt{(x-c_x)^2+(y-c_y)^2}}{R}$
3. Blend:
   $\text{Out} = \text{In}(1-d) + \text{Blur} \cdot d$

Single pass. $O(W \times H)$.









Adar's Filters

[Emboss] Relief Emboss

Computes the difference between top-left and bottom-right neighbors, offset by 128 to produce a 3D gray-relief texture.

[Swirl] Swirl

Polar-coordinate warp: each pixel's radius drives a nonlinear angular rotation before being mapped back.

[Dodge] Dodge

Applies the Color Dodge formula per channel: $\text{Out} = \frac{p \times 255}{\max(1,\, 255 - p)}$
Blows out highlights, raises contrast sharply.

[Bloom] Bloom

Extracts pixels above intensity 180, applies a 7×7 brute-force blur, and additively composites the glow onto the original.

Deniz's Filters — Overview

[Threshold] Threshold

Luminance $L = 0.299R + 0.587G + 0.114B$. Pixels above threshold $T \rightarrow$ white; otherwise black.

[Crosshatch] Crosshatch

Procedural sketch rendering using repeating mathematical line patterns to simulate pencil hatching.

[Diffuse] Diffuse

Frosted-glass effect: each pixel is replaced by a randomly sampled neighbor within a fixed radius.

[Hue] Hue Rotate

HSV-space hue shift:
$H' = (H + \Delta H) \bmod 180$.
$\Delta H = 60$ preserves geometry and brightness.

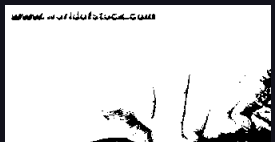
Threshold & Diffuse Results

Threshold

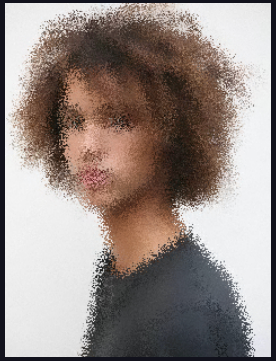




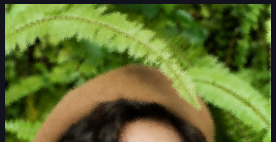


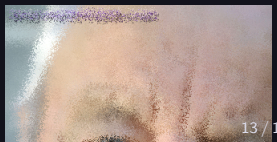


Diffuse



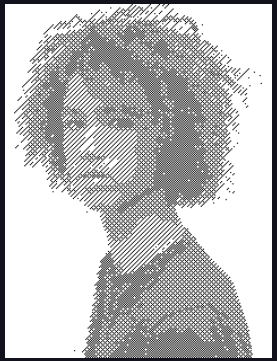




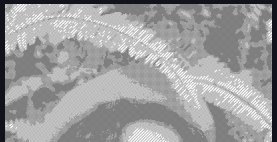


Crosshatch & Hue Rotate Results

Crosshatch



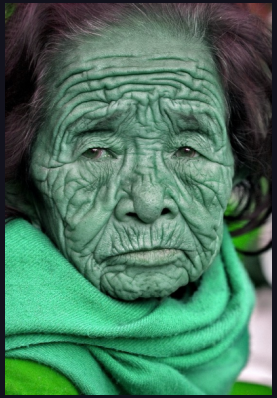


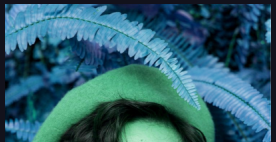




Hue Rotate









Summary

What We Built

- 12 manually implemented image filters
- Interactive OpenCV GUI with live preview
- Pixel-level C++ — no high-level filter APIs
- Modular architecture (one header per filter)

Key Takeaways

- All filters run in $O(W \times H)$ time
- LUTs improve per-pixel cost to array lookups
- YCrCb space prevents color drift in contrast ops
- Inverse mapping avoids holes in geometric

Filter Breakdown

Author	Filters
Eyobed	Hist Eq, Lens Dist, Posterize, Vignette
Adar	Emboss, Dodge, Swirl, Bloom
Deniz	Threshold, Diffuse, Crosshatch, Hue Rot

Thank You

Questions?

Adar Otieno Deniz Aksoy Eyobed Awel Nuri

ZPOe 25/26 — Brno University of Technology